Presentation Transcript (5 minutes)

Review 3 deck · Adaptive CPU Scheduling Using Predicted Burst Lengths
Shivaansh Aggarwal (24BCE0883) · Devansh Sharma (24BCE0717)

Shivaansh: slides 1, 2 (first half), 3, 4, 7, 9, 10, 11. **Devansh:** slides 2 (second half), 5, 6, 8. *Italics = what to do, not what to say.* The slide numbers match **Review_3.pptx**.

1 – Title 15 s

Shivaansh

Good morning ma'am. I'm Shivaansh, and this is Devansh. Our scheduler **predicts how long a program will run next** and uses that to decide **who runs and for how long**. In this review we also asked **what happens when the prediction is wrong**.

2 – The problem and our approach 40 s

Shivaansh

SJF is optimal but **needs the future**, and Round Robin's quantum is **fixed by hand**. Predicting bursts fixes that, but a predicting scheduler **trusts its prediction**: a long burst predicted as short **keeps looking short** every time it comes back from a time slice.

Devansh

So we **use the prediction twice**: to pick **who runs**, with ageing, and to set **how long it runs** from the 80% rule. And we added an **overrun guard**: once a burst clearly outruns its prediction, its estimate **grows with the CPU time it has used**.

3 – How we evaluated it 25 s

Shivaansh

7 workloads, 14 configurations, 10 evaluation seeds, all paired. Training, tuning and evaluation used **separate seeds**, every claim is checked with **paired tests and a Holm correction**, and **53 tests pass**. **W7 is deliberately adversarial**: bait processes use **76% of the CPU**.

4 – Main result 25 s

Shivaansh

On the mixed workload we wait **27% less than SJF**, better on **all 10 seeds** and **significant**. We close **54% of the gap to the oracle** and use **61% fewer switches** than Round Robin with 5 ms.

5 – Across the six standard workloads 30 s

Devansh

The clear win is the **mixed workload**, and the **worst case drops by about half** on CPU-bound and phase-changing work. The quantum adapts to each workload, and prediction alone gives only 5%. Honestly, on **all-I/O work we are 14% worse**, and that loss is significant.

6 – How much prediction quality matters 30 s

Devansh

We fed our scheduler and plain SJF predictions with **controlled error**. With perfect predictions our mechanism still beats SJF by **28%**, so **the mechanism matters most**. Perfect predictions would give another **22%**, and **real errors hurt more than random ones**.

7 – The weakness 30 s

Shivaansh

A bait process runs tiny bursts, so the model predicts **1.2 ms**, then it runs for hundreds. Without a guard it keeps **looking like 1.2 ms** and is picked again. The guard sets its time left to **a quarter of the CPU time used**, never less than the prediction: after 40 ms it looks like **10 ms**.

8 – Live demo on W7 35 s

Devansh

Optional: run `python demo_innovation.py` in Terminal (about 10 s); otherwise point at the screenshot.

Without the guard we **lose to Round Robin and MLFQ**. With it, over ten seeds, the average wait drops **21.5%** and interactive waiting **92%**. But **MLFQ still has a 9% lower average** on W7, and our **worst case is about twice theirs**.

9 – Trade-offs and a negative result 30 s

Shivaansh

On the five standard workloads the guard changes the average by **less than 2%, none significant**. We also made the ageing credit persistent; it raised waits by **17% under overload and 51% on W7**, so **we did not keep it**.

10 – Comparison with existing schedulers 20 s

Shivaansh

On the mixed workload we beat every classical scheduler except SRTF, which is **3% better but not significantly**, with a **35% higher worst case**. On W7, **MLFQ has the lowest average**; with the guard we beat Round Robin.

11 – Conclusion and future work 20 s

Shivaansh

Using the prediction twice gives **27% less waiting**, mainly thanks to the mechanism. The guard fixes **a real weakness of prediction-based scheduling**, but it is **not a complete defence**. Next: **real traces** and **xv6**. **Thank you, ma'am.**

Total: about 5 minutes.

If asked – one-line answers

Why ML? (Devansh) **12–22% more accurate** than exponential averaging, but the sweep shows **the mechanism matters more**.

What if the prediction is wrong? (Devansh) That is what **W7 and the guard** test: interactive waiting **−92%**, but MLFQ still wins on average there.

Is W7 fair? (Shivaansh) It is **deliberately adversarial**, not typical; we say so, and the guard was **tuned on seeds 100–102** and tested on 0–9.

Is it significant? (Shivaansh) Paired tests on 10 seeds with **Holm correction**; we only call a result significant if $p < 0.05$ after correction.

Real OS? (Shivaansh) It's a **controlled simulation**; a kernel would need a smaller model, and **xv6** is next.